(NRAI) Agentic Platform Primitive IDD

Basic Definitions (Nouns)
Agentic Platform Core Capabilities
New Architecture
Capabilities Exposed via APIs
Object Definitions
User Journey
Agent Observability
Use-cases to be developed as part of Agentic Platform (3-4 months)
Choice of multi-agent framework
Knowledge Graph
Small Language Models
World Models (Next evolution and future direction)

Basic Definitions (Nouns)

1. **Tool**: Tools empower agents to interact with external data and services while unlocking a wider range of actions beyond that of the underlying model alone. Tools can take a variety of forms and have varying depths of complexity, but typically align with common web API methods. For example, a tool could update customer information in a database or fetch weather data to influence a travel recommendation that the agent is providing to the user. In short, *tool does a task*.
2. **Agent**: An application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal. Agents are autonomous and can act independently of human intervention, especially when provided with proper goals or objectives they are meant to achieve. Agents can also be proactive in their approach to reaching their goals. Even in the absence of explicit instruction sets from a human, an agent can reason about what it should do next to achieve its ultimate goal. In short, *an agent is a planner of tasks to achieve a goal*.
3. **Knowledge Graph:** A knowledge base that uses a graph-structured model to represent the data and operate on it. Historically associated with search engines like Google, Bing and social networks like Linkedin, Facebook and question-answering services like Alexa, Siri. They are used to store interlinked descriptions of objects or events.
   Disclaimer: While we believe creating knowledge graphs on NR's internal data will help us in a variety of tasks like better question-answering or incident response, this can be a good addition to Orchestrator engine or specific Agents and is not a core platform component. So, we move the discussion on Knowledge graph out of this document.
4. An **orchestrator agent** is a component within a system, often used in distributed computing and automation environments, responsible for coordinating and managing the execution of tasks or processes across multiple agents or services. The orchestrator acts as a central controller to ensure that complex workflows are executed efficiently, reliably, and in the correct sequence.
5. A **routing agent** is a component within a system that directs data or tasks to the appropriate destinations based on predefined rules or dynamic conditions
6. **Workitem** is a lightweight ticket that represents an action to be taken by the customer. WorkItems can sync with the customer's ticketing system. They also are the input to workflows
7. A **workflow** can be any type of action that has steps (manual or automated) between start and finish. Examples of workflows include new user signup, production change approval, synthetics, polling cloud providers for telemetry, and alerting
8. **NRAI vs Agentic Platform**: NRAI is superset of Agentic Platform. Agentic Platform will enable developers to build agents and tools. These developers could be internal to NewRelic, our partners, or even customers. It will also enable customers to configure, and use those agents. Customers will also be able to trigger agents based on conditions. In addition to the Agentic Platform, NRAI will also enable aspects like LLMs/SLMs, ML Models, etc.

Agentic Platform Core Capabilities

Agentic Platform is the core platform that provides common capabilities required to build Agentic orchestration for all New Relic teams, our partners and our customers. Platform provides highly scalable capabilities, such as (this list is not conclusive, and will evolve as the product and newer usecases emerge) -

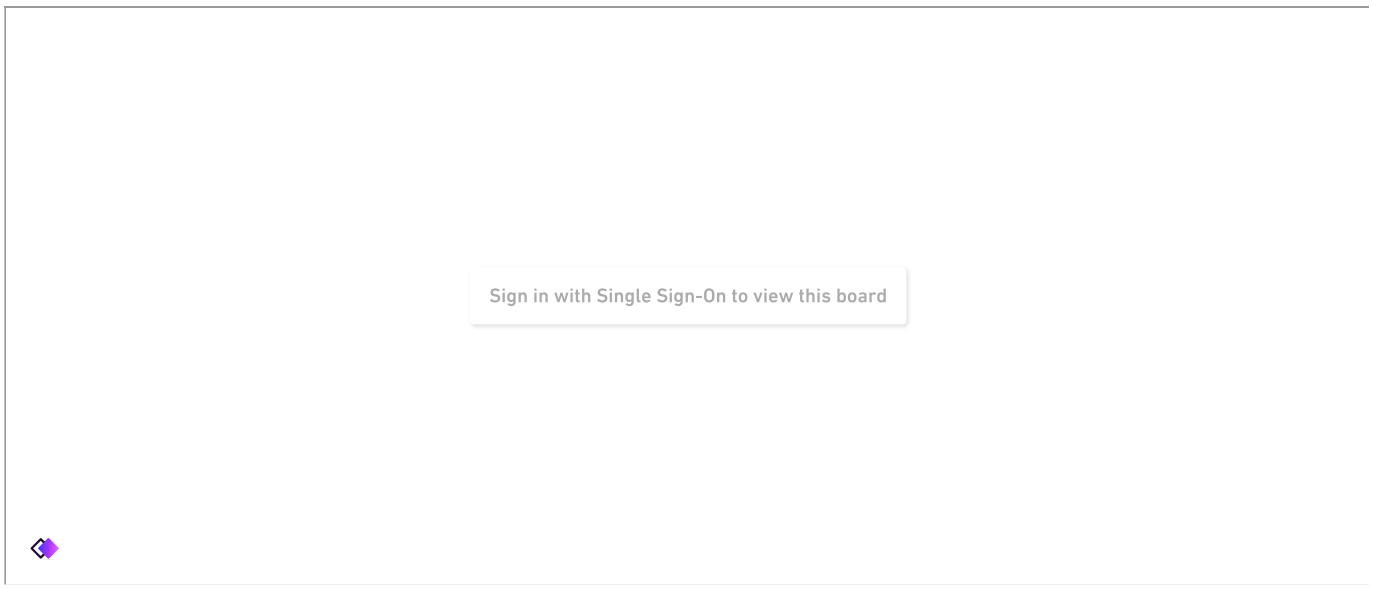
- Declarative way for the developers to define/validate/execute the agents and tools. Developers could be NR teams, external partners or customers.

- Library of prebuilt agents and tools
- Agentic Orchestration - key to how universal Agents are hosted on the platform and orchestrate to accomplish extremely complex workflows. Our orchestrator agent(s) would also be capable of orchestrating external agents (partner or customer) and receiving orchestration instructions/plans from partner or customer orchestrators.
- APIs, Agent and Tool Development - key to make Agentic Platform usage simple and standardized
- External Integrations - critical for us to scale our platform. We will win by strong partnerships where we can iterate with our partners quickly. We should engage with an open, robust ecosystem of agents and orchestrators.
- Platform Efficiencies - Agentic platform will be responsible for running at high scale with high SLAs both from performance, reliability and COGS perspective. Running Agents at high scale are extremely costly and unless we have a central place to manage cost effectively, we will run into margin challenges
- Agent observability - Platform will allow observability of agent behaviour and performance
- Agent evolvability - Agents should be able to evolve based on the customer feedback

In addition to the above agentic platform capabilities, NRAI platform also provides below capabilities

- RAG Platform and Connectors to plug data from custom sources
- Knowledge Graph leveraging inputs from established New Relic sources, such as entity relationships from entity platform, “workloads” from Transaction 360, RAG, and others to build rich insights within New Relic universe that ground agent reasoning and analysis
- Core SLMs and ‘World’ models - this is important for very precise use of Language Models and evolving New Relic’s detection and prescription capabilities
- Generative UX - AI experience has to permeate every part of New Relic experience. Our UX should be adaptive, dynamic, and problem-solution scoped
- Traditional ML use case like Anomaly Detection, predictions, forecasting, classification, etc
- Experimentation framework to test most effective SLM/LLMs and ~~Agent grouping~~
- Model evolvability - Models should be able to evolve based on the customer feedback

New Architecture



Key Decisions:

1. While there is no standard for the **agentic programatic interface**, OpenAI interface is widely adopted by multiple LLM providers. With than in mind, we have an intention to support OpenAI interface. OpenAI interface is REST based, but the NewRelic recommended interface is GraphQL based. NerdGraph(NR implementation GraphQL) automatically handles aspects like auth. To take care of compatibility as well as NR standards, NRAI will primarily support NerdGraph based interface. But to take care of compatibility with the OpenAI SDK, NRAI will build a compatibility service which will be further interacting with NerdGraph.

2. One of the basic expectation from platform is evolution, i.e. in our case how can agents and tools evolve without negatively impacting customer. With that in mind, platform will need to support **version management of agents and tools**, i.e.
   a. There could be multiple versions of an agent deployed. But there would only be one live version of an agent which is based on developer's decision. e.g. if there is `codeReview` agent, with two versions `v1` and `v2`. Then there will be two deployed agents `codeReview@v1` and `codeReview@v2`. And customer or developer could use specific version of the agent. But if just `codeReview` agent is used, we will use `codeReview@v1` or `codeReview@v2` based on which one has developer marked as live.
   b. Developer of agents/tools and push multiple versions and test those before making it live
   c. Customers could use latest live version of the agents/tools or pin their usage to a specific version of the agent/tool
   d. Platform will enable developers to do A/B testing of different versions of agents/tools, and accordingly observe its errors/feedback before flowing 100% traffic to a new version.
3. Every developer of agent is expected to provide **declarative agent definition** containing:
   a. Prompt to be used for executing the agent
   b. Tools available to the agent
   c. LLM to be used for the agent
   d. ...
4. All the routing to the required agent will happen in the **orchestrator engine**. This engine would also load multiple agents and tools, and multiple versions of each using the multi-agent framework of the choice. This multi-agent framework isn't yet decided and could be of the likes of LangGraph, Crew.ai, etc. Orchestrator engine will also use tools like LLMGuard to protect the ingestion attacks and other attack vectors as per OWASP. The below agents are crucial in Agentic platform
   a. **Orchestrate Agent** is responsible for coordinating multiple agents or processes to achieve specific objectives. It acts as a central controller that manages task execution and resource allocation across the system.
   b. **Planner Agent** is focused on devising strategies to achieve long-term goals. It often deals with generating plans involving sequences of actions or selecting optimal paths to desired outcomes.
   c. **Specialist Agent** is designed to perform specific tasks requiring specialised knowledge or skills. It acts as an expert in a particular domain or function.
   d. **Critic agent** continuously evaluates actions or decisions made by other agents or the system itself, providing feedback that can be used to improve future performance.
5. **Validation pipeline** is the one responsible for validating agents or tools when they are created/updated by the developers. These validations will happen based on the bunch of tests as provided by the agent developer. This is important to ensure continuous quality of agents irrespective of changes in agents, or slm/llm, or data sources down the line.
6. There is a need for **long term memory** that stores the context of users. This context can be used by the agent to decide the response and execution path based on the context. This context is also used to build **user profile** and accordingly stored in the same long term memory. This profile can be used by agents to tweak prompts based on the user profile.
7. In the long term, platform should be able to train its own SLM and store in the internal artifactory for usage by the agents. Having said that, platform should be open enough to support external LLM/SLM like OpenAI, Gemini, self-hosted open source LLMs, etc based on the agent needs as well as the customer security needs, e.g. some customers might not like chat messages going outside NewRelic network.
8. Agents built on the Agentic Platform could use multiple data sources(NRQL, NerdGraph, websearch, external endpoint, vector store, etc). Similarly these agents could trigger an action (e.g. trigger autoflow, mutation request to NerdGraph, etc). If it is left to the tools to write code to make such calls to the outer world, it will lead to duplication of many concerns in all tools, e.g. error handling, retry, auth, rate limiting, etc. Also we would prefer tools to be defined using just metadata, instead of logic on how to fetch/update data. With this in mind, we will be introducing new service in NRAI, i.e. **Data Sources and Actions Service**. This service will be still outside the Agentic Platform. This service will act as an adapter, rather than proxy to the services down the line. It would receive metadata from tools and accordingly act on it by trigger the services down the line.

> Above architecture doesn't talk about interaction to the externally deployed agents. It will need to be brainstormed further among the engineering team. @Manpreet Singh (Deactivated) @Akhilesh Chander Myakala @Sreekanth Tadakaluru @Vijay Ram Sevagapandian to collaborate further on the same.

Capabilities Exposed via APIs

1. Basic CRUD APIs of NGEP for entities:
   a. ToolDefinition
   b. Tool
   c. AgentDefinition
   d. Agent
   e. Config/Map
2. API to call Validation Pipeline (Tool, Agent meets criteria)
3. API to Agentic Interface (Live Orchestration)

Object Definitions

0. **Interface: Entity** (as defined in NgEP)

Base Entity Interface

```
interface Entity {

  "The entity's global unique identifier."
  id: ID! @readonly

  "The entity type."
  type: String! @readonly

  "The entity's scope (enum - Organization/Account)"
  scope: ScopedReference! @final

  "The name of the tool"
  name: String!

  "Metadata about the entity (has fields- createdAt, updatedAt, createdBy)"
  metadata: Metadata! @readonly

  "Collection of tags (key-value pairs)"
  tags: [Tag!]
}
```

1. **AgenticTool**
   Def: An AgenticTool is a "Tool in GenAI terminology" which just '*does a task*'. The tasks can be of a wide variety like converting NL (Natural language) to NRQL query or doing a simple correlation analysis or opening a Jira ticket. However these tools can be 'broadly' classified into data retrieval tools, analytical tools and action tools.

   A tool can point to an endpoint among the hosted models or 3rd party API endpoints (like Azure OpenAI) of different LLM, SLM models that we provide through the platform. One cannot add new models, but can configure tools/agents to use a specific model.

   The mutations allowed in the public API is only CREATE and UPDATE, but it will be the responsibility on developer to not update the live tool. The same applies for other object types here.

AgenticTool Entity

```
//For the developer to define a tool
type AgenticToolDefinitionEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){

  "{..fields of Interface Entity..}"

  "The key attributes specific to the tool"
  attribute: {
    description: String!
    category: enum! //"The key skill category of the tool - eg Data Retrieval, Analysis, Forecasting etc."
    customAttribute: CollectionEntity //"Collection identifier of the entity as defined by the developer"
    argumentOrder: [Argument] //Mentions the arguments in a specific order as neeeded by the tool
    configRequired: [Config] //Json schema of all the config parameters
  }

  "Link to the tool's location (http endpoint or the link to python object)"
  location: String!


  "Link to LLM/SLM model"
  model: {
     endpoint: String, //http endpoint or link to python object
     secretKey: String
  }
```

```
  "Monotonically increasing version number"
  version: Int! @final

  "Identifies if created by an internal user or the external user"
  "visibility(product)"
  sourceIdentifier: enum! @internal
  }


//For developer to be able to mark a specific version live among multiple versions she created
type AgenticToolEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){
   liveToolDefinition : ID! //Identifier of a Tool Definition
}

type Config {
            "name": String!,
            "description": String,
            "tool": [String],
            "isRequired": Boolean,
            "default": Int
}

type Argument {
            "name" : String,
            "description": String
}
```

2. **AgenticAgent**

   Def: An AgenticAgent is an “Agent in GenAI terminology” which uses LLM’s intelligence to plan and implement and achieve an objective. It uses tools (Type: AgenticTool) to implement the various tasks as per the plan. These have a similar classification (skill category) like AgenticTool and also have a pointer to LLM/SLM model.

AgenticAgent Entity

```
type AgenticAgentDefinitionEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){

  "{..fields of Interface Entity..}"

  ""The key attributes specific to the agent"
  attribute: {
    description: String!
    prompt: String
    category: enum! //"The key skill category of the tool - eg Data Retrieval, Analysis, Forecasting etc."
    customAttribute: CollectionEntity //"Collection identifier of the entity as defined by the developer"
    configRequired: [AgenticAgentDefinitionConfig] //Json schema of all the config parameters
  }

  "Identifier of Orchestrator Agent"
  isOrchestrator : Boolean!

  "Identifiers of the AgenticTools that the Agent needs to be mapped to"
  toolids: [ID!]

  "Identifiers of the other AgenticAgents that this Agent needs to be mapped to"
  agentids: [ID!]

  "Conditional Link to the next Tool/Agent"
  conditionalEdge: {
     value: Boolean,
     conditionalAgentIds: [ID]!
  }

  "Link to LLM/SLM model"
  model: {
     endpoint: String, //http endpoint or link to python object
     secretKey: String
  }

  "Agentic Framework to be used"
  framework: Enum! //Langgraph or Autogen or CrewAI

  "Monotonically increasing version number"
  version: Int! @final

  "TestCase set identifier"
  testCase: ID!

  "Identifies if created by an internal user or the external user"
  "visibility(product)"
  sourceIdentifier: enum! @internal

  "Dependency on external APIs - Yes/No"
  "visibility(flag:AgenticAI/ngTypes)"
  dependencyFlag: enum!
  }
```

```

//For developer to be able to mark a specific version live among multiple versions she created
type AgenticAgentEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){
   liveAgentDefinition : ID! //Identifier of an Agent Definition
   weight : Int! //Integer between 1 to 100 mentioning % of weight given to this tool
}

type AgenticAgentDefinitionConfig {
            "name": String!,
            "description": String,
            "tool": [String],
            "isRequired": Boolean,
            "default": String
}
```

Undirected (Acyclic/Cyclic) graphs of agentic workflows can be represented by above schema. But for Directed graphs, we will have to make modifications to data structures of toolids, agentids, which is an open item for now.

3. **AgenticMap**

   Def: The mapping between an Agent and a list of Tools (or) an Agent and other Agents can be changed with this configuration (Note that there is a map configuration that a developer chooses while developing an agent, but this can be changed by a customer using this Map).

AgenticMap Entity

```
type AgenticMapEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){

 "{..fields of Interface Entity..}"

  "The description of the Config"
  description: String!

  "Identifier of the AgenticAgent in the Config Change"
  agentid: ID!

  "Identifiers of the AgenticTools that the Agent needs to be mapped to as an alternative to original Agent Config"
  toolMap: [ID!]

  "Identifiers of the AgenticAgents that the Agent needs to be mapped to as an alternative to original Agent Config"
  agentMap: [ID!]

  "Conditional Link to the next Tool/Agent"
  conditionalEdge: {
     value: Boolean,
     conditionalAgentIds: [ID]!
  }

  "Config as required by the AgenticAgentDefinition"
  config: [AgenticMapConfig]

}

type AgenticMapConfig {
  "name": String!,
  "value": String!
}
```

4. **AgenticTestCases**:

Def: This is a set of TestCases used by Validation pipeline in the platform to validate a Tool or an Agent.

AgenticTestCases Entity

```
type AgenticTestCasesEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){

 "{..fields of Interface Entity..}"

  "The description of the Set"
  description: String!

  "test case array which will be executed in the sequence."
  testCases: [TestCase!]!
}

type TestCase {
  input : String!
  output : String!
}
```

5. **AgenticTrigger**

   This entity type will be created by the customers to create trigger to define the conditions on workitems based on which one or more agents can be triggered.

AgenticTrigger Entity

```
type AgenticTriggerEntity implements Entity @mutations(mutationTypes: [CREATE, UPDATE]){

 "{..fields of Interface Entity..}"

  "The description of the trigger"
  description: String!

  "Type of the workItem"
  workItemType: [String!]

  "Conditions based on the agents will be triggered"
  conditions: [Condition!]

  "Identifiers of the AgenticAgents that the Agent needs to be mapped to as an Override of original Agent Config"
  agentsToTrigger: [ID!]
}

type Condition{
...
}
```

Above is under the assumption that agents would be triggered based on the changes in the work items, which would be internally stored in NGEP.

There is an open question on how would same agent be configured multiple times, e.g. using different communication channels for each agent execution.

User Journey

Sign in with Single Sign-On to view this board

```
Summary:

Step 1 : See how is my use-case currently addressed by the Platform and retrieve the configuration of Agents and Tools used
         //One can also retrieve the entities by an ID or a string search - the usual Nerdgraph support
Step 2: See if the new tool I developed meets the requirements of the platform
Step 3-6: CRUD APIs provided by the NGEP (for Tool, Agent, Map)
Step 7: Update my entities as needed and request Platform to deprecate my older versions

const nerdgraphEndpoint = 'https://one.newrelic.com/graphql';

//Step 1 - Send a inference request to "Agentic Interface" and observe the tools and agents invoked
             mutation {
                 callAgenticInterface($input: JsonPayloadInput, $tags:[Tag]) {
                    runid, //Identifier of the result object
                    [EntityId] //Entities
                }
              }
              query:
```

```
                getAgentRun($runid: id) {
                   ....
                      }
        //Fetch the information on each of tools, agents involved in the above call and observe their configurations
            query:
              actor {
                EntityManagement {
                  entity(id: "1234-5678-9000...") { //Similar query for Tool, Agent, Map
                     id,
                     scope,
                     name,
                     type,
                     tags, //tags can store the variety of input-output pair examples that are addressed by the Tool/Agent
                     metadata
              }

        (//Assume the developer finds an opportunity to develop a new tool here)
//Step 2 - Find the suitability of my tool by calling Validation Pipeline
            createTestCases{}
            mutation {
                doValidation($AgenticTool: JsonPayloadInput, $tags:[Tag]) {
                   runid //Returns id of the result object - having a single Boolean field indicating whether it meets or doesn't meet compliance
                }
              }
            //Behind the scenes -
            //Checks requirements like Atomicity of Tool (not a straightforward task),
            //Also - Cohesion between input and outputs as claimed, redundancy in platform etc.
//Step 3 - createAgenticTool (Optional) // Requirements will be provided (in the docs). Eg: Single Responsibility Principle, Atomicity, Cohesion
             mutation {
                 createAgenticTool(
                 {
                  name: "NL2NRQL",
                  scope: {
                    type: "ORGANIZATION",
                    id: "1" //unique identifier of the object within given scope
                  },
                  description: "This converts Natural language prompts to NRQL query to be used to query New Relic Data base"
                  ...
                  //default LLM model
                }
              )
            }
        //Behind the scenes - checks if there is a similar tool and throws a soft error
//Step 4 - createAgenticAgent (Optional)
             mutation {
                 createAgenticAgent(
                 {
                  name: "DataRetrievalAgent",
                  scope: {
                    type: "ORGANIZATION",
                    id: "2" //unique identifier of the object within given scope
                  },
                  toolIds: [123, 345],
                  description: "This Agent has intelligence to pull the data using the most relevant tool among various tools like NL2NRQL, NL2Nerdgraph, RAG etc."
                  ...
                  //default LLM model
                }
              )
            }
           createRelationship(newAgeticTool, newAgenticAgent)
//Step 5 - createAgenticMap (Optional)
             mutation {
                 createAgenticMap(
                 {
                  name: "DataRetrievalMap",
                  agentId: 123,
                  toolIds: [456, 789, 1010], //Changes the mapping created during Agent
                }
              )
            }
           createRelationship(newAgenticAgent, newAgenticMap)
//Step 6 - Repeat step 1 to check the response from the new agent
//Step 7 - Create AgenticTrigger to trigger agents based on the changes in the workitem
            mutation {
                 createAgenticTrigger(
                 {
                  name: "TriggerforIncidentManagement",
                  workItemType: Jira,
                  condition: [
                    "key": "status",
                    "value": "open"
                  ],
                  agentstoTrigger: "AgentMap1"
                }
              )
            }
//Step 8 - Create an updated tool/agent or/and make a request to Platform to deprecate the earlier version
        //Behind the scenes - Platform team requests all users to shift their Maps from older tool to newer tool
        //Once the Maps using the original tool reduce to zero, it will be deprecated privately
```

Agent Observability

Each of the below personas requires different kind of agent observability:

- Agent developer
  - Usage Metrics at agent level
  - Error Metrics at agent level
- NR support/employee
  - Metrics at tool/agent/platform level
- Customers of NRAI
  - Traces of whole chat communication
  - LLM generated labels for a conversation
    i. Topic Cluster
    ii. Resolution Status
    iii. Time to Resolution (not LLM generated)
    iv. Reason for Resolution Status

Use-cases to be developed as part of Agentic Platform (3-4 months)

Refer to the [master steel-thread document]() for more details. Why these usecases ?

1. **Immediate relevance**: Each scenario addresses **urgent, high-friction tasks** that SREs and DevOps teams face almost daily (post-deployment checks, major incident firefighting, and postmortem clean-up or gameday-ing).
2. **Agentic approach**: All three showcase the **full agentic cycle**—observing, interpreting, planning, acting, and continuously improving—rather than just providing static recommendations.
3. **Scalable augmentation**: They scale to thousands of alerts or changes, effectively granting teams a "virtual SRE staff" without costly new hires.
4. **RAG & feedback loops**: By leveraging Retrieval-Augmented Generation, each agent can consult historical context, best practices, and user feedback to **improve performance over time**, aligning with the vision of "an SRE on contract" that learns with every incident.

1. Post-Deployment "Closed-Loop" Monitoring Agent

Use Case 1

Scenario and motivation

When a new change is deployed (whether a code update, config modification, or infrastructure change), an SRE typically must ensure that:

1. Targeted monitoring has been set up or updated.
2. Metrics collected after the change can be measured against baselines.
3. Alert conditions are created or updated to detect potential issues with that particular release.
4. Error budgets or performance thresholds are watched/monitored to inform rollback decisions.

Pain points

- Managing alerts and thresholds is often time-consuming (if manual) or inaccurate (if Terraform is used, for example, to set alerts as part of the release process)
- Poorly correlated alerts post-deployment flood on-call engineers, create noise.
- Teams don't always clean up "temporary" stuff after concluding the release validation window (alerts, dashboards, etc.).

Closed-loop Deployment Monitoring Agent steel thread

A **closed-loop deployment monitoring agent** automates and orchestrates post-deployment monitoring and rollback readiness and activities, effectively serving as an extra SRE you would "hire" to do the repetitive, time-sensitive tasks.

1. **Data gathering / watching**

- Integrated with change tracking (springs into action when change event is recorded) and queries for past change events to look for similar past events
- Uses RAG to retrieve relevant historical data on similar deployments or known issues.
- Pulls baselines, performance metrics from (MELT, etc.) from right before the change.
- Forecasts signals after change

2. **Interpretation**
   - Calculates a new **short-term error budget** (e.g., 5-minute or 30-minute windows) to reflect the updated entity's SLO constraints (from New Relic Service Levels).
   - Contextualizes gathered data via entity relationships to set the release's risk profile
3. **Planning**
   - Dynamically creates or updates alert conditions (or synthetic checks) based on the new release's risk profile.
   - Adjusts the frequency of its own re-queries (e.g., more frequent checks in the first 30 minutes post-deployment) based on trends, forecasts
4. **Action**
   - Spins up dashboards, sets custom alert thresholds, and triggers synthetic tests.
   - If thresholds are violated, **auto-initiates** a rollback procedure or provides a recommended rollback plan.
   - Consolidates newly triggered alerts into a single "incident thread" on collaboration platform (Slack, Teams) rather than spamming multiple tickets.
5. **Quality Management**
   - After the monitoring window, automatically **cleans up** temporary alert rules and dashboards.
   - Logs results (e.g., MTTD, MTTI, rollback success/failure) for continuous learning.
   - Provides a short summary report or triggers a Jira ticket with relevant details.

Value

- **Reduces repetitive manual tasks** for SREs.
- **Minimizes mean time to detect and repair** post-deployment issues.

Builds confidence in frequent deployments by providing an always-on "safety net."

2. Augmented War Room / Turmoil Agent

Use Case 2

Scenario and motivation

When major production incidents or large events occur (e.g., new market launches, holiday traffic spikes, or major marketing campaigns), the SRE on-call or a lead engineer often scrambles to gather insights from numerous dashboards and logs. They may also have to coordinate with multiple teams—often in real-time war-room scenarios.

Pain points

- Humans can't watch **all** metrics simultaneously in high-pressure events.
- **Escalation** often depends on guesswork or heroics.
- L1 engineers or generalist on-call staff lack deep expertise, requiring numerous escalations.

War Room Agent steel thread

A **"War Room Augmentation Agent"** becomes a virtual extension of the on-call team, automatically serving as an "extra set of eyes/hands" (scanning key metrics, correlating alerts, and suggesting real-time corrective actions) during incident response.

1. **Data gathering / watching**
   - Continuously checks in on a curated "crucial signals list" (e.g., CPU usage, key transaction latencies, error rates) specified in runbooks or given by responders upon agent initiation.
   - Integrates topology data (Service Maps, Workloads, Transactions 360) to establish range it must traverse as it watches for issues e.g. the entire stack from infrastructure to service level.
2. **Interpretation**
   - Uses **anomaly detection, forecasting** plus event correlation to identify the most critical performance or availability issues.

- Consults a RAG platform for historical context on similar incidents—what signals mattered or not, which actions were effective or not.

3. **Planning**
   - Intelligently decides which part of the stack or which entity it should assess next based on analysis of results so far
   - Continuously prepares and updates recommended next steps (e.g., scale out a specific microservice, throttle requests from a problematic region).
4. **Action**
   - If authorized, can **auto-scale** resources or temporarily block traffic from a suspect endpoint.
   - Escalates to the correct subject-matter experts via Slack/Teams, including a summary of relevant metrics and logs.
5. **Quality Management**
   - Captures actions taken and outcomes (e.g., "Auto-scale succeeded in reducing latency by 40%").
   - Updates knowledge base (RAG, KG) with "lessons learned," improving detection and escalation logic for next time.

Value

- Provides **real-time "extra sets of eyes"** on critical metrics, entity relationships.
- **Reduces war-room fatigue** and guesswork by systematically feeding new information to engineers who are working in parallel to pinpoint the source of an issue
- Allows L1 or general on-call staff to handle more complex incidents with confidence, freeing senior SREs/heroes for deeper tasks.

3. We need to put the GH Copilot SRE agent workflow here - `@Camden Swita` please add.

Choice of multi-agent framework

Choosing the right multi-agent framework is critical for the successful development and deployment of agentic systems. There are 3 popular frameworks.

1. **LangGraph** is an open-source framework by Langchain designed for building stateful, multi-agent applications using large language models (LLMs). It models workflows as graphs, with each node representing a specific task, enabling detailed control over application flow and state. This approach is ideal for complex workflows requiring advanced memory features, error recovery, and human interaction, and it integrates seamlessly with LangChain tools and models.
2. **Autogen**, developed by Microsoft, is a framework for creating conversational agents that treat workflows as conversations, similar to ChatGPT interfaces. It supports tools like code executors and function callers, enabling agents to handle complex tasks autonomously. The framework is highly customisable and modular, making it suitable for both simple and complex multi-agent scenarios.
3. **CrewAI** is a framework that enables collaborative role-based AI agents, each assigned specific roles and goals to function together as a unit. It's ideal for creating advanced multi-agent systems, like research teams, by supporting flexible task management and autonomous inter-agent delegation. CrewAI also provides customizable tools, enhancing its versatility in multi-agent collaborations.

At present, the experiments are leaning towards Autogen. We plan to conduct a thorough evaluation of all the frameworks.

Detailed comparison

Criteria	LangGraph	Autogen	CrewAI	Final Verdict
Ease of Usage	✗	✓	✓	Autogen and CrewAI are more intuitive due to their conversational approach and simplicity.
Multi-Agent Support	✓	✓	✓	CrewAI excels with its structured role-based design and efficient interaction management among multiple agents.
Tool Coverage	✓	✓	✓	LangGraph and CrewAI have a slight edge due to their extensive integration with LangChain.
Memory Support	✓	✓	✓	LangGraph and CrewAI are advanced in memory support features, ensuring contextual awareness and learning over time.
Structured Output	✓	✓	✓	LangGraph and CrewAI have strong support for structured outputs that are versatile and integrable.
Documentation	✓	✓	✓	LangGraph and CrewAI offer extensive and well-structured documentation, making it easier to get started and find examples.
Multi-Agent Pattern Support	✓	✓	✓	LangGraph stands out due to its graph-based approach, which makes it easier to visualize and manage complex interactions.
Caching	✓	✓	✓	LangGraph and CrewAI lead with comprehensive caching mechanisms that enhance performance.
Replay	✓	✗	✓	LangGraph and CrewAI have inbuilt replay functionalities, making them suitable for thorough debugging.
Code Execution	✓	✓	✓	Autogen takes the lead slightly with its innate code executors, but others are also capable.
Human in the Loop	✓	✓	✓	All frameworks provide effective human interaction support and are equally strong in this criterion.
Customization	✓	✓	✓	All the frameworks offer high levels of customization, serving various requirements effectively.
Scalability	✓	✓	✓	All frameworks are capable of scaling effectively, recommend experimenting with each to understand the best fit.
Open source LLMs	✓	✓	✓	All frameworks support open-source LLMs.

Knowledge Graph

Expand to view details

A knowledge graph in the context of observability is a structured representation of data and relationships that enhances the understanding, analysis, and management of digital systems.

Knowledge graphs in observability help in transcending traditional monitoring by providing meaningful insights through an interconnected dataset, which aids in optimising performance, enhancing reliability, and ensuring that systems meet desired operational objectives.

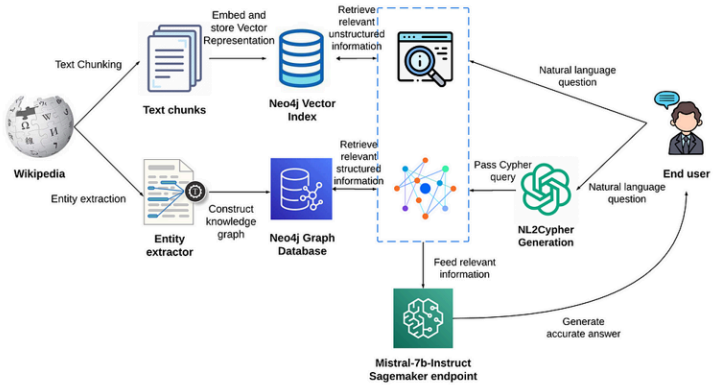
Potential Data Sources

1. Logs
2. Metrics
3. Traces
4. Events
5. Alerts, Incidents and Configuration Changes
6. Metadata

Use Cases

1. Root Cause Analysis
2. Performance Optimisation
3. Impact Analysis
4. Security and Compliance
5. Intelligent Alerting and Reporting

Copied Diagram. Working on creating our NR specific architecture.



Small Language Models

Expand to view details

Small Language Models (SLMs) are a subset of language models that are designed with fewer parameters compared to large, resource-intensive models like GPT-3 or GPT-4. Despite having a smaller size, they aim to perform natural language processing (NLP) tasks effectively while being more efficient in terms of computational resources and energy consumption.

In the context of observability, fine-tuning or creating small language models can enhance the ability to process, interpret, and act on system data. This can improve monitoring, debugging, and overall system performance. Here are some specific observability use cases, along with data sources and preparation strategies.

Potential Data Sources

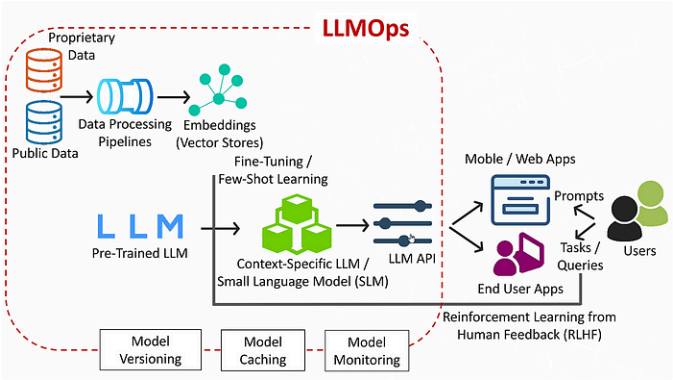
1. Logs
2. Metrics
3. Traces
4. Events
5. Alerts, Incidents and Configuration Changes
6. Metadata

Use Cases

1. Log Analysis and Anamoly detection
2. Incident Response
3. Predictive maintenance (World Models)
4. Reporting
5. NL2NRQL
6. RCA
7. Health Monitoring of systems

Data Preparation Strategies

1. Data Cleaning (Normalising the log formats, remove unwanted information,Data consistency)
2. Data Annotation
3. Data Augmentation
4. Balancing and Sampling
5. Feature Engineering
6. Tokenisation and embedding



World Models (Next evolution and future direction)

Expand to view details

Ref : World Models for Autonomous Driving: An Initial Survey

A world model is a cognitive framework used in artificial intelligence (AI) and robotics to represent an understanding of the environment in which an agent operates. It provides a structured representation of the various elements and dynamics of a given environment, allowing the agent to make predictions, plan actions, and adapt to changes.

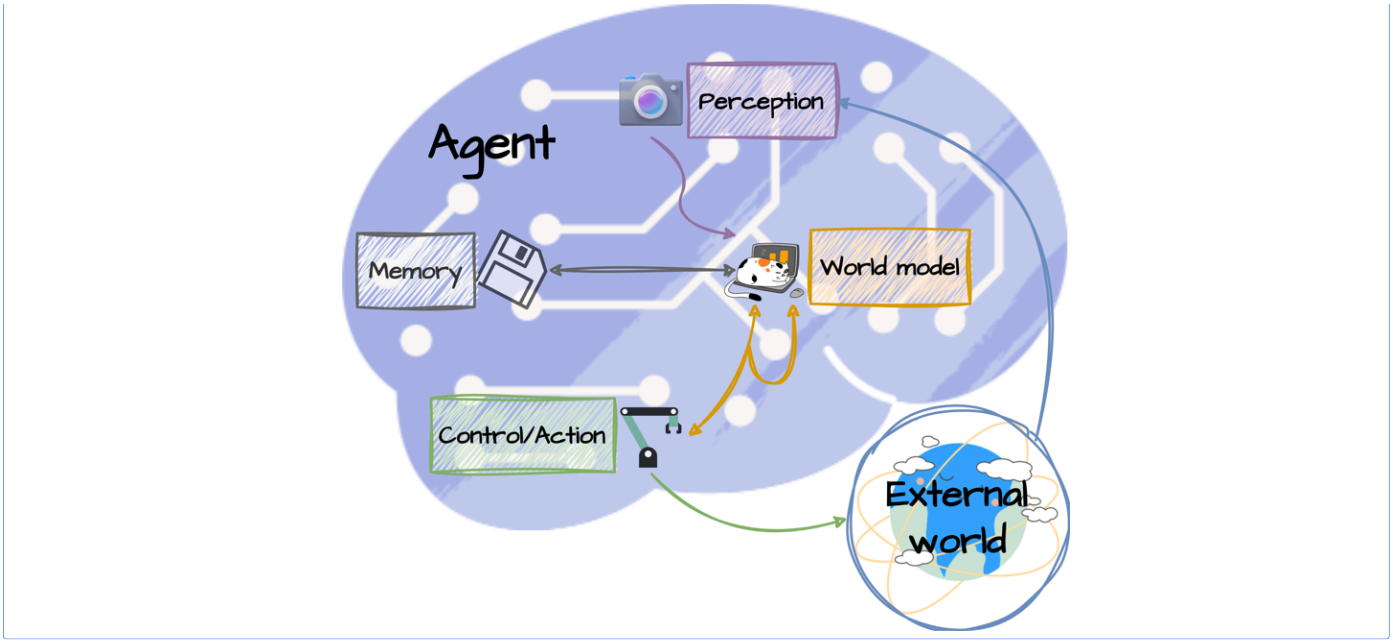
In the context of observability, "world models" refer to computational models that represent, understand, and predict the state of a system or environment by integrating and processing various sources of observational data. These models aim to encapsulate the dynamic behavior of a system over time and provide comprehensive insights into its operations

Key Components of World Models in Observability

1. Data Integration
2. State Representation
3. Temporal Dynamics
4. Predictive modelling
5. Scenario Simulation

Use Cases of World Models

1. Anomaly Detection
2. Capacity Planning and Resource Optimisation
3. Root Cause Analysis and Troubleshooting
4. Adaptive Systems and Automation
5. Impact Analysis


Agent
Perception
Memory
World model
Control/Action
External world